

Otomotiv Yazılım Mimarisi ve Yapay Zeka Destekli Geliştirme Süreçleri: Kapsamlı Strateji ve Proje Yönergeleri

Otomotiv endüstrisi, tarihinin en büyük teknolojik kırılmalarından birini yaşamaktadır. Yüz yılı aşkın süredir mekanik ve termodinamik prensiplerle şekillenen araç tasarımı, günümüzde "Yazılım Tanımlı Araçlar" (Software-Defined Vehicles - SDV) paradigmasıyla tamamen baştan yazılmaktadır. Geleneksel E/E (Elektrik/Elektronik) mimarilerinde birbirinden bağımsız çalışan, her biri tek bir spesifik işlevi yerine getiren ve sinyal tabanlı protokollerle haberleşen dağıtık Elektronik Kontrol Üniteleri (ECU), yerini yüksek işlem gücüne sahip alan kontrolcülerine (Domain Controllers) ve merkezi araç bilgisayarlarına (High-Performance Computing - HPC) bırakmaktadır¹. Bu radikal dönüşüm, otomotiv yazılım işine profesyonel ve ciddi bir ölçekte giriş yapmayı hedefleyen girişimler için benzersiz bir fırsat penceresi sunmaktadır.

Bu kapsamlı araştırma raporu; elektrikli, dizel ve benzinli araç türlerinin yazılım gereksinimlerini, mimari temel taşı olan AUTOSAR (Classic ve Adaptive) platformlarını, uluslararası güvenlik regülasyonlarını (ISO 26262, UNECE R155/R156) ve bu karmaşık mühendislik süreçlerinin modern yapay zeka ajanları (özellikle Claude Code ve otonom kodlama araçları) ile nasıl entegre edileceğini endüstriyel bir derinlikle incelemektedir. Süreçlerin salt bir kod yazım faaliyeti olmaktan çıkarılıp, "Bağlam Mühendisliği" (Context Engineering) ve Model Tabanlı Geliştirme (Model-Based Development) prensipleriyle nasıl kurgulanacağı eksiksiz bir biçimde ortaya konmuştur.

Güç Aktarım Sistemlerine Göre Otomotiv Yazılım Mimarileri

Otomotiv yazılım projelerinde temel katman mimarisi, aracın tahrik sisteminin (powertrain) türüne göre radikal farklılıklar gösterir. Her bir güç aktarım organı; sensör veri toplama mekanizmaları, algoritmik sinyal işleme süreçleri ve aktüatör kontrol dinamikleri açısından benzersiz matematiksel modellere ve gerçek zamanlı işlem kısıtlamalarına tabidir. Geliştirilecek yazılımın, motor bloğu veya batarya paketi içindeki fiziksel ve kimyasal reaksiyonları mikrosaniyeler seviyesinde yönetmesi gerekmektedir.

Benzinli Araçlar: İçten Yanmalı Motor Yönetim Sistemleri (EMS)

Benzinli motor (Spark Ignition - SI) kontrol yazılımlarının birincil hedefi, yanma verimliliğini maksimize ederek spesifik yakıt tüketimini düşürmek ve emisyon değerlerini global regülasyon sınırları içinde tutmaktır. Bu süreç, yakıt enjeksiyon sürelerinin ve ateşleme avansının (ignition timing) devir ve yük haritalarına göre anlık olarak hesaplanmasını gerektirir⁴. Elektronik Kontrol Ünitesi (ECU) yazılım mimarisi, kütle hava akış sensöründen (Mass Air Flow Sensor) gelen voltaj veya frekans değişkenlerini hava kütle akış oranına çeviren spesifik yazılım bileşenleri (MassAirFlowSensorSWC) üzerine inşa edilir⁷.

Oksijen (Lambda) sensörlerinden alınan geri bildirimler kullanılarak, hava-yakıt oranının (Air/Fuel Ratio) ideal stokiyometrik noktada ($\Lambda = 1$) tutulması, üç yollu katalitik konvertörün verimli çalışması için kritik bir yazılım döngüsüdür⁴. Motor kontrol yazılımı; soğuk çalıştırma, tam gaz ivmelenme (WOT - Wide Open Throttle) ve rölanti gibi farklı çalışma koşulları için hem açık döngü (open-loop) hem de kapalı döngü (closed-loop) algoritmalar barındırır⁸. Rölanti devri yönetimi, motor soğutma suyu sıcaklığına ve klima kompresörü gibi ek yüklerle göre yazılım tarafından dinamik olarak hesaplanarak hedeflenen devre ulaşmak üzere sürekli bir geri bildirim mekanizması işletir⁸.

İleri mühendislik uygulamaları, fiziksel sensörlere olan bağımlılığı azaltmak için model tabanlı yazılım algoritmalarına yönelmektedir. Sensörsüz lambda operasyonları, enjeksiyon miktarındaki bilinen bir değişimin motorun çalışma karakteristiğine (örneğin volan ivmelenmesine) etkisini analiz eden adaptif algoritmalar sayesinde güncel hava-yakıt karışımını sanal olarak tahmin edebilmektedir¹⁰. Bu tür matematiksel modeller, motor pompalama kayıplarını (pumping losses) azaltan ve kısmi yük (POT) operasyonlarında verimi artıran yenilikçi ön-oda ateşleme sistemleri (prechamber ignition systems) ile entegre çalıştığında, geleneksel yazılım çözümlerine kıyasla önemli bir teknolojik sıçrama yaratır⁹. Yazılımın validasyonu aşamasında, Eddy akımı dinamometreleri (eddy current dynamometer) ile sağlanan kapalı döngü yükleme sistemleri sayesinde, motorun karakteristik tork ve güç eğrileri laboratuvar ortamında simüle edilerek yazılım algoritmalarının tepki süreleri ölçümlenir⁴.

Dizel Araçlar: Sıkıştırma Ateşlemeli Motor ve Emisyon Yönetimi

Dizel motor yazılımları, benzinli motorlardan farklı olarak, buji ateşlemesi yerine yüksek basınçlı sıkıştırma prensibine dayandığından, yakıt dağıtım (Common Rail) dinamiklerine ve çok daha katı hale gelen partikül emisyon kontrollerine odaklanmak zorundadır. Bosch EDC15C7 gibi sektör referansı motor kontrol üniteleri, enjektör açılma sürelerini, yüksek basınç pompasını ve turbo şarj (VNT/VGT) basıncını entegre bir şekilde yönetir¹¹.

Dizel araç yazılımlarında işlemci gücünün aslan payını egzoz gazı yönetimi ve son işlem (after-treatment) algoritmaları tüketir. Azot oksit (NOx) emisyonlarını düşürmek için kullanılan Egzoz Gazı Devridaimi (EGR) sisteminin valf açıklığı oranları, sensör verileriyle beslenen haritalar üzerinden dinamik olarak hesaplanır¹¹. Aynı şekilde, Dizel Partikül Filtresi (DPF) içerisindeki kurum miktarını diferansiyel basınç sensörleriyle hesaplayan algoritmalar, filtre doluluk oranı kritik eşiğe ulaştığında enjeksiyon stratejisini değiştirerek egzoz sıcaklığını artıran ve "Rejenerasyon" modunu tetikleyen durum makineleri (state machines) kullanır¹¹. Ek olarak, Seçici Katalitik İndirgeme (SCR) sistemlerinde, egzoz hattına püskürtülecek AdBlue (üre) miktarının, egzoz gazı sıcaklığı ve anlık NOx yoğunluğu baz alınarak milisaniyeler içinde optimize edilmesi gerekmektedir¹¹.

Elektrikli Araçlar (EV): Batarya, Güç Elektroniği ve Termal Dinamikler

Elektrikli araç yazılımları, otomotiv yazılım mühendisliğini geleneksel termodinamik kontrollerden uzaklaştırarak, elektrokimyasal sistemlerin, karmaşık güç elektroniğinin (inverter kontrolü) ve termal yönetim sistemlerinin idaresine evriltir. Bu araçların merkezi yazılım bileşeni olan Batarya Yönetim Sistemi (BMS - Battery Management System), yüzlerce pil hücresinden oluşan yüksek

voltajlı paketlerin güvenliğini ve verimini sağlamakla yükümlüdür¹².

BMS yazılımlarında algoritmik derinlik genellikle Şarj Durumu (State of Charge - SoC) hesaplamaları ile başlar. Akım entegrasyonu (Coulomb counting) ve açık devre voltajı ölçümleri kullanılarak bataryanın anlık enerji seviyesi tahmin edilir¹³. Ancak asıl mühendislik zorluğu, bataryanın kimyasal ömrünü ve iç direnç artışını modelleyen Sağlık Durumu (State of Health - SoH) algoritmalarının geliştirilmesidir¹³. Batarya paketindeki her bir hücrenin voltaj ve sıcaklık değerleri bağımsız olarak izlenmeli, şarj-deşarj sırasında hücreler arasında oluşan voltaj dengesizliklerini gidermek için pasif veya aktif hücre dengeleme (cell balancing) algoritmaları koşturulmalıdır¹³. Yazılım tabanlı bu güvenlik fonksiyonları; aşırı ısınma, aşırı şarj veya derindeşarj durumlarını saniyenin kesirleri düzeyinde tespit edip kontaktörleri açarak bataryayı izole eden arıza güvenliği (fail-safe) mekanizmalarıyla donatılmak zorundadır, aksi takdirde tehlikeli termal kaçak (thermal runaway) vakaları ortaya çıkabilir¹².

Otomotiv Yazılım Mimarisinin Çatısı: AUTOSAR Platformları

Dünya çapında farklı OEM'lerin (BMW, Ford, Toyota vb.) ve Tier-1 tedarikçilerin (Bosch, Continental) ürettiği donanım ve yazılımların birbirleriyle sorunsuz entegre olabilmesi için otomotiv endüstrisi, AUTOSAR (Automotive Open System Architecture) standardını benimsemiştir². Elektrikli, dizel veya benzinli fark etmeksizin; fren, direksiyon, hava yastığı gibi güvenlik açısından kritik sistemlerle, otopilot ve bilgi-eğlence gibi esnek ve dinamik sistemler arasında mimari bir ayrım yapılması zorunludur. AUTOSAR standartları, bu ayrımı Classic ve Adaptive olmak üzere birbirini tamamlayan iki farklı mimari platform ile kurgular¹.

AUTOSAR Classic: Gerçek Zamanlı ve Deterministik Sistemler

2003 yılında standardize edilen AUTOSAR Classic, yüksek güvenilirlik gerektiren sert gerçek zamanlı (hard real-time) uygulamalar için tasarlanmış statik, derleme zamanında (compile-time) yapılandırılan bir mimaridir¹. Sınırlı bellek ve işlemci gücüne (Flash kapasitesi ≤ 4 MB, RAM ≤ 512 KB) sahip mikrodenetleyiciler (MCU) üzerinde çalışır ve şasi, güç aktarma organları, fren kontrolü gibi alt sistemleri hedefler¹.

Klasik mimari belirli katmanlara bölünerek donanım bağımlılığını en aza indirir:

- **Mikrodenetleyici Soyutlama Katmanı (MCAL):** Çipe özel donanım sürücülerini (ADC, SPI, CAN, FlexRay) barındıran ve donanıma doğrudan erişen en alt katmandır¹.
- **ECU Soyutlama Katmanı:** Harici cihazlara (örneğin harici EEPROM'lar) donanımdan bağımsız erişim sunar¹.
- **Temel Yazılım (BSW - Basic Software):** OSEK tabanlı statik İşletim Sistemi (OS), iletişim yığını (COM, PDU Router), bellek yönetimi (NvM) ve teşhis (DCM/DEM) servislerini sunar¹.
- **Çalışma Zamanı Ortamı (RTE - Runtime Environment):** Uygulama katmanındaki yazılım bileşenlerini (SWC) alt katmanlardan izole eden, port tabanlı iletişimi sağlayan dinamik olarak üretilmiş orta katmandır¹.
- **Uygulama Katmanı (Application Layer):** Gerçek kontrol algoritmalarının, sensör okuma ve eyleyici sürme mantığının çalıştığı kısımdır.

Bu mimarinin belirleyici özelliği statik olmasıdır. Yazılım bileşenleri (SWC) arasındaki tüm bağlantılar, mesajların hangi CAN frame'ine yerleştirileceği ve işletim sistemi görevlerinin zamanlaması, derleme işlemi öncesinde XML tabanlı ARXML dosyalarında tanımlanır¹. Çalışma zamanında (runtime) dinamik servis keşfi veya yeni bir görevin başlatılması söz konusu değildir; her şey önceden hesaplanmış ve belirlenmiştir¹.

AUTOSAR Adaptive: Yüksek Performans ve Hizmet Odaklı Mimari

Bağlantılı araçlar (connected cars) ve otonom sürüş (ADAS) sistemlerinin getirdiği yüksek işlem gücü gereksinimi, klasik mikrodenetleyicilerin kapasitesini aşmıştır. Bulut tabanlı hizmetlerle haberleşme, kamera/radar/LiDAR gibi yüksek bant genişliği tüketen sensör verilerinin işlenmesi ihtiyacı 2017 yılında AUTOSAR Adaptive platformunun doğmasını sağlamıştır¹.

Adaptive platform, Linux veya QNX gibi POSIX uyumlu işletim sistemleri üzerinde çalışan, mikroişlemci (SoC/MPU) odaklı bir yapıdır¹. C yerine Nesne Yönelimli (Object-Oriented) bir yaklaşım getiren C++14/17 dilleri kullanılarak geliştirilir¹. Klasik mimarideki statik, sinyal tabanlı (CAN/LIN) iletişim modelinin yerini, çalışma zamanında dinamik olarak keşfedilen Hizmet Odaklı Mimari (Service-Oriented Architecture - SOA) ve Ethernet tabanlı SOME/IP veya DDS protokolleri alır¹.

Adaptive mimari, yazılım tanımlı araçlar için hayati olan iki önemli konsepti devreye sokar:

1. **Güncelleme ve Konfigürasyon Yönetimi (UCM):** Uygulamaların aracın yaşam döngüsü boyunca Havadan (Over-The-Air - OTA) güvenli bir şekilde güncellenmesini, yeni özelliklerin eklenmesini ve silinmesini yönetir¹.
2. **Yürütme Yönetimi (Execution Management):** Adaptive uygulamaların çalışma zamanı boyunca başlatılması, duraklatılması ve sonlandırılması süreçlerini POSIX mimarisinin sağladığı çoklu işlem (multi-threading) yetenekleriyle yönetir¹.

Mimari Boyut	AUTOSAR Classic	AUTOSAR Adaptive
Donanım Hedefi	Mikrodenetleyiciler (Infineon TC3xx, NXP S32K) ¹	SoC'ler ve MPU'lar (NXP S32G, Qualcomm SA8xxx) ¹
İşletim Sistemi Altyapısı	AUTOSAR OS (OSEK Tabanlı, Statik, Minimal) ¹	POSIX Uyumlu (Linux, QNX) ¹
İletişim Paradigması	Sinyal Tabanlı, Sabit Yönlendirmeli (CAN, LIN, FlexRay) ¹	Servis Odaklı, Dinamik Keşif (SOME/IP, Ethernet) ¹
Konfigürasyon	Tamamen Statik, ARXML ile Derleme Zamanı ¹	Dinamik, Manifest dosyaları ile Çalışma Zamanı ¹
Programlama Dili	C (Sıkı MISRA-C Kuralları) ¹	C++ (C++14 / C++17)

		standartları) ¹
Fonksiyonel Güvenlik	Kanıtlanmış ASIL D yetkinliği ¹	Gelişmekte olan ASIL D, ağırlıklı QM - ASIL B ¹
Tipik Uygulama Alanları	Motor kontrolü (ECU), Şasi, Fren sistemleri, Hava yastığı ¹	Otonom Sürüş (ADAS), Bilgi Eğlence, Telematik, V2X ¹

Model Tabanlı Geliştirme (MBD) ve Sistem Entegrasyonu

Otomotiv yazılımlarında, özellikle klasik mimari kapsamında karmaşık kontrol algoritmalarını doğrudan donanım seviyesinde elle kodlamak (hand-coded C) güvenlik ihlallerine ve tasarım uyumsuzluklarına yol açma riski taşır¹⁷. Model Tabanlı Geliştirme (MBD) yaklaşımı, mühendislere matematiksel işlevleri MATLAB/Simulink veya ETAS ASCET gibi görselleştirilmiş blok tabanlı platformlar üzerinden tasarlama imkanı verir¹⁷.

Süreç, Yukarıdan Aşağıya (Top-Down) bir metodoloji izler; sistem seviyesindeki spesifikasyonlar, yazılım bileşenleri, arayüzler ve "runnable" görevler olarak modellenir¹⁴. Geliştirilen bu modeller, TargetLink veya Embedded Coder gibi üretim kod oluşturucuları (code generators) aracılığıyla otomatik olarak ISO 26262 uyumlu C koduna dönüştürülür¹⁷. Modellerin davranışı ile üretilen kodun hedef donanımdaki davranışı arasındaki eşdeğerliği ispatlamak için Model-in-the-Loop (MiL), Software-in-the-Loop (SiL) ve en nihayetinde Hardware-in-the-Loop (HiL) testleri (back-to-back testing) uygulanır¹⁸. Ayrıca Silver gibi sanal ECU (Virtual ECU) teknolojileri sayesinde, gerçek donanım prototipleri üretilmeden aylar önce kapalı döngü simülasyonları gerçekleştirilebilir¹⁷.

Fonksiyonel Güvenlik ve Siber Güvenlik Regülasyonları

Otomotiv yazılım endüstrisinde yazılan her satır kod, insan hayatını doğrudan etkileme potansiyeline sahiptir. Modern bir yolcu aracı, yirmi yıl önceki ticari bir uçaktan daha fazla elektronik kontrol ünitesi barındırmakta ve bu karmaşıklık yepyeni hata vektörlerini beraberinde getirmektedir¹⁶. Bu nedenle proje yönetimi, kodlama kalitesinden bağımsız olarak katı endüstriyel regülasyonlar üzerine inşa edilmelidir.

ISO 26262 ve Fonksiyonel Güvenlik (Functional Safety)

ISO 26262, karayolu taşıtlarında kullanılan elektrik/elektronik (E/E) sistemlerin fonksiyonel güvenliğini sağlamak için tasarlanmış, tüm ürün geliştirme yaşam döngüsünü kapsayan bir standarttır¹². Standart, gereksinim analizinden sistem konseptine, donanım-yazılım geliştirme evresinden entegrasyon ve doğrulama süreçlerine kadar ilerleyen klasik "V-Modeli" mühendislik döngüsünü zorunlu tutar¹². V-Modelinin özü, sol taraftaki her bir tasarım ve tanımlama aşamasının, sağ tarafta karşılık gelen bir doğrulama ve test aşamasıyla eşleştirilmesine dayanır¹². Sürecin belkemiği, konsept aşamasında yürütülen Tehlike Analizi ve Risk Değerlendirmesi (HARA - Hazard Analysis and Risk Assessment) adımıdır¹². Olası her bir tehlike senaryosu üç temel

parametre üzerinden puanlanır:

1. **Şiddet (Severity - S):** Meydana gelecek fiziksel zararın boyutu. S_0 (yaralanma yok) düzeyinden S_3 (hayatı tehdit eden veya ölümcül kaza) seviyesine kadar sınıflandırılır¹⁶.
2. **Maruz Kalma (Exposure - E):** Tehlikeli durumun gerçekleşme olasılığı veya sıklığı. E_0 (gerçekleşmesi neredeyse imkansız) ile E_4 (yüksek olasılıklı) arasında değer alır¹⁶.
3. **Kontrol Edilebilirlik (Controllability - C):** Tehlike anında sürücünün veya sistemin aracı güvenli bir şekilde yönlendirebilme kabiliyetidir. C_0 (genel olarak kontrol edilebilir) ile C_3 (kontrol edilmesi imkansız) arası derecelendirilir¹⁶.

Bu faktörlerin birleşimi, her bir güvenlik hedefi için bir Otomotiv Güvenlik Bütünlük Seviyesi (ASIL - Automotive Safety Integrity Level) belirler¹⁶. ASIL A en düşük risk seviyesini ifade ederken, ASIL D en yüksek fonksiyonel güvenlik kısıtlamalarını temsil eder; QM (Kalite Yönetimi) ise standart dışı risksiz modüller için kullanılır¹².

ASIL Seviyesi	Otomotiv Sistem Örnekleri	Yazılım Mimari ve Test Zorunlulukları (Part 6)
ASIL D	Hava Yastığı Ateşlemesi, Elektrikli Direksiyon (EPS), Elektronik Fren Sistemi (Brake-by-wire) ¹² .	%100 MC/DC (Modified Condition/Decision Coverage) kodu doğrulama zorunluluğu ¹⁶ . Mimari düzeyde sıkı donanım/yazılım izolasyonu ve yedeklilik. Yazılım için statik analiz mutlak gerekliliktir ¹⁸ .
ASIL B / C	Adaptif Hız Sabitleyici, Elektronik Stabilite Kontrolü, İleri Sürüş Destek (ADAS) frenleme algoritmaları ¹⁶ .	Branch (Dal) Coverage testleri, sağlam bellek yönetimi (Memory Protection) ve MISRA C standartlarına zorunlu uyum ¹⁶ .
ASIL A	Gövde kontrolleri (Korna, fren harici sinyalizasyon ışıkları) ¹⁶ .	Informal doğal dil gereksinim yönetimi, temel Statement Coverage kod testleri yeterli kabul edilir ¹⁶ .

QM	Klima Sistemi (HVAC), Bilgi-Eğlence Ekranları, Elektrikli Koltuklar ¹⁶ .	Standart şirket içi kalite süreçleri uygulanır. Özel ISO 26262 güvenlik döngüsüne tabi tutulmazlar ¹⁶ .
----	---	--

Yazılım geliştirme katmanında (ISO 26262 Part 6), yüksek riskli modüllerin donanım maliyetini dengelemek adına ASIL Ayrıştırması (Decomposition) uygulanabilir. Örneğin; sistemin bütüncül olarak ASIL D seviyesini sağlaması gereken bir durumda, gereksinimler tamamen bağımsız ve ortak arıza noktası (common-cause failure) taşımayan iki farklı donanım mikroişlemcisine ASIL B(D) olarak bölüştürülebilir¹⁶.

C ve C++ dillerinin doğasında var olan tanımlanmamış davranışları (undefined behavior) engellemek amacıyla kodlama evresinde MISRA C:2023 ve MISRA C++:2023 kılavuzlarına katı biçimde uyulması ISO 26262 tarafından zorunlu kılınır¹⁸. Makine Öğrenimi (ML) ve Yapay Sinir Ağları (DNN) içeren otonom sistemlerde eğitilmiş verinin yerel optimizasyon farklılıkları (instability) yaratması, geleneksel V-Modelinin öngördüğü belirlenimcilik (determinism) felsefesine ters düştüğünden, standardın makine öğrenimi bileşenlerinin yalnızca birim (komponent) seviyesinde sınırlandırılmasını tavsiye etmesine neden olmuştur²⁰.

Siber Güvenlik (UNECE R155/R156 ve ISO/SAE 21434)

Bağlantılı araç mimarilerinin yaygınlaşması, bilgi-eğlence sistemi üzerinden fren veya direksiyon ağlarına erişim sağlanabilmesi ihtimalini (lateral movement) beraberinde getirerek siber güvenliği hayati bir konu haline getirmiştir²². Birleşmiş Milletler Avrupa Ekonomik Komisyonu (UNECE) WP.29 düzenlemeleri, tüm araç üreticilerini (OEM) sertifikalı güvenlik süreçlerine tabi tutmaktadır²². Avrupa Birliği pazarında Temmuz 2024 itibarıyla zorunlu olan bu regülasyonlar, Türkiye Cumhuriyeti Sanayi ve Teknoloji Bakanlığı tarafından (SGM-2022/13 tebliği vb.) ulusal mevzuata aktarılmış ve TSE (Türk Standardları Enstitüsü) bünyesinde uyumlandırılmıştır²³.

1. **UNECE R155 (Siber Güvenlik Yönetim Sistemi - CSMS):** OEM'lerin ve alt yüklenici yazılım tedarikçilerinin araç elektroniğini tehditlere karşı güvence altına alacak bir süreç inşa etmesini gerektirir²³. Donanım Emniyet Modülü (Hardware Security Module - HSM) kullanımı ve kriptografik anahtarların izolasyonu standart bir uygulama haline gelmektedir²⁵.
2. **UNECE R156 (Yazılım Güncelleme Yönetim Sistemi - SUMS):** Havadan yapılan güncellemelerin (OTA) güvenliğinin sağlanmasıdır. Güncelleme paketlerinin dijital olarak imzalanması, yetkisiz yazılımların araca yüklenmesinin engellenmesi ve yeni versiyonun, var olan güvenlik (ASIL) performansını hiçbir koşulda geriye götürmediğinin regülatif otoriteler nezdinde ispatı zorunludur²³.
3. **ISO/SAE 21434:** R155'in yönetsel zorunluluklarını teknik bir mühendislik standardına dönüştürür. Geliştirme yaşam döngüsü içerisine "Tehdit Analizi ve Risk Değerlendirmesi" (TARA) eklenerek, yazılım zafiyetlerinin (vulnerability analysis) tasarım aşamasında yok edilmesi hedeflenir²⁴.

Yapay Zeka Ajanları İle Proje Yürütme: Strateji ve

Mimari

Otomotiv yazılımı geliştirmek, binlerce satır C/C++ dosyasını, spesifik ARXML konfigürasyonlarını, donanımsal register tanımlamalarını ve sıkı güvenlik kurallarını aynı anda yönetmeyi gerektirir. Böylesi bir projeyi yapay zeka entegrasyonu ile yönetirken, klasik ChatGPT benzeri sohbet asistanlarını (chatbot) kullanarak kodu kopyala-yapıştır yöntemine başvurmak, çok kısa sürede bağlam (context) kaybına, bozuk derleme (build) süreçlerine ve tehlikeli entegrasyon hatalarına neden olur³⁰.

Bunun yerine, doğrudan kod geliştirme ortamında çalışan, dosya sistemini okuyup değiştirebilen ve terminal komutlarını çalıştırabilen "Ajan İş Akışları" (Agentic Workflows) entegre edilmelidir³⁰.

Doğru Ajan Aracı Seçimi: Aider vs Cline vs Claude Code

Piyasada yazılım otomasyonu için tasarlanmış birbirinden farklı felsefelere sahip güçlü otonom araçlar bulunmaktadır³¹. Bu projenin yürütülmesinde kullanılacak AI entegrasyon modeli şu karşılaştırmalara göre seçilmelidir:

- **Aider (Git Tabanlı Otonomi):** Terminal ortamında çalışan açık kaynaklı bir araçtır. En büyük gücü Git sistemini derinden anlamasıdır. Yapay zeka ajanının her mantıksal değişikliği otomatik olarak "conventional commit" mesajlarıyla versiyonlaması ve herhangi bir hata durumunda "undo" (geri alma) özelliği sayesinde koda geri dönme esnekliği sunması, yoğun refaktör işlemleri için idealdir³².
- **Cline ve Roo Code Çatallamaları (Görsel Onaylı Workflow):** VS Code tabanlı eklentilerdir (Roo Code orijinal desteği Mayıs 2026'da kesilmiş, forkları olan Zoo Code veya Cline popülerleşmiştir). En ayırt edici özelliği, yapay zekanın kod üzerinde değişiklik yapmadan veya terminalde bir betik çalıştırmadan önce, mühendisten görsel bir arayüz (GUI) üzerinden her bir adımı onaylamasını (approval) beklemesidir³⁰. Ayrıca alt-ajanlar (subagents) sayesinde farklı görevlerin (örneğin; aynı anda test yazımı ve arayüz çizimi) paralel ilerlemesine izin verir³².
- **Claude Code (Anthropic'in Otonom Ajanı):** Büyük ölçekli ve yüksek bağlam gerektiren (1 Milyon tokene kadar) bir otomotiv projesi için en tutarlı araç olarak öne çıkar³⁰. Terminalde yaşar ve otonom döngü kurma kabiliyeti en yüksek olan ajandır³⁰. Yazılımın, saatler boyunca terminalde test derleyicilerini (Make/CMake) çalıştırıp çıkan derleme hatalarına göre kendi kodunu düzelterek (self-correction) kapalı bir döngüde ilerlemesine olanak tanır³⁴.

Projeyi Claude asistanı ile birlikte yürütebilmek için, temel IDE ortamınıza (VS Code, Cursor vb.) terminal yetkilerine sahip Claude Code aracı entegre edilmeli ve projenin "Bağlam Mühendisliği" (Context Engineering) kurgulanmalıdır³⁰.

Bağlam Mühendisliği ve CLAUDE.md Tasarımı

Yapay zeka ajanı, proje dizinini incelediğinde motor yönetim sisteminin veya ISO 26262 kurallarının bilincinde değildir³⁷. Claude'un hatalı mimari kararlar vermesini (halüsinasyon), yanlış kütüphaneler kullanmasını ve şirket standartlarından sapmasını önlemek adına, kök dizinde CLAUDE.md isimli bir yapılandırma dosyası (yapay zekanın anayasası) oluşturulması kritik önem

taşıır³⁴. Claude her oturuma başladığında bu dosyayı gizli bir sistem direktifi olarak okur ve modelleme davranışını buna göre şekillendirir³⁴. Endüstriyel tecrübeler, bu dosyanın token israfını ve dikkat dağınıklığını önlemek adına 300 satırın altında, son derece net kurallar kümesinden oluşması gerektiğini göstermektedir³⁷.

Aşağıda, planlanan otomotiv sistemi projesi için optimize edilmiş bir CLAUDE.md yapısı yer almaktadır:

Proje Genel Bakış (Project Overview)

Bu repo, Elektrikli (BMS), Dizel (EMS/EDC15C7) ve Benzinli araç platformları için AUTOSAR standartlarında C ve C++ dillerinde yazılım bileşenleri (SWC) içerir. Tüm bileşenler, insan hayatını etkileyebilecek fonksiyonel güvenlik kısıtlarına tabidir. Önceliğimiz sıfır derleme hatası, sıfır MISRA ihlali ve deterministik kod yapısıdır.

Klasör Hiyerarşisi ve Mimari (Architecture)

- /bsw: AUTOSAR Basic Software katmanı (OS, Com, Mem) statik dosyaları¹
- /rte: Runtime Environment dinamik üretilen ara yüzleri (dokunma)¹
- /swc_classic: ASIL C/D mikrodeneleyici algoritmaları (BMS Hücre dengeleme, Motor enjeksiyonu)¹
- /swc_adaptive: QM/ASIL-B seviyesinde POSIX/C++ ADAS ve Telematik sistemleri¹
- /tests: MiL/SiL test suite konfigürasyonları¹⁸

Katı Kurallar ve Teknoloji Yığını (Tech Stack & Rules)

- TIER 1 RULE: Classic platform (/swc_classic) içinde SADECE MISRA C:2023 standartlarına uyulacaktır. Dinamik bellek yönetimi (malloc(), free()) kullanımı KESİNLİKLE yasaktır¹⁸.
- Adaptive platformda (/swc_adaptive) Modern C++ (C++14) ve SOME/IP iletişim kütüphaneleri kullanılacaktır¹.
- Yeni SWC tasarlanırken önce ARXML dokümantasyonu üretilecek, ardından kodlama yapılacaktır.

Test ve Doğrulama Şartları (Quality Bar)

- TDD (Test-Driven Development) metodolojisi zorunludur. Kod yazılmadan önce test dosyası yazılmalı, testin bilerek başarısız olduğu gösterilip sonra uygulama kodlanmalıdır⁴⁰.
- ISO 26262 ASIL D gereği, tüm modüller Branch Coverage ve MC/DC (Modified Condition/Decision Coverage) analizlerinden geçmelidir¹⁶.

Bu dosyanın gücünü artırmak için "Hook" (Kanca) mekanizmaları devreye alınır⁴⁰. Claude Code, terminalde kodu derledikten sonra, örneğin statik kod analiz aracını (MISRA kontrolcüsü) tetikleyen bir pre-commit hook sayesinde kural ihlali yapan kodu versiyon deposuna gönderemez ve kendi hatasını düzeltmek için mecburen otonom bir geri dönüş (retry) döngüsüne girer⁴⁰.

Ajan Döngüsü: Planlama ve TDD Yaklaşımı

Otomotiv yazılımı doğrudan (zero-shot) yaklaşımla kodlanamaz. Projede Claude Code ile ilerlerken her özellik isteği (örneğin "BMS SoC algoritmasını yaz") **Plan Modu** kullanılarak başlatılmalıdır³⁰.

1. **Araştırma ve Planlama (Shift+Tab):** Claude Code'a görev verildiğinde doğrudan kod yazması engellenir. Model öncelikle repo içerisindeki sensör başlık dosyalarını okur, RTE haberleşme portlarını analiz eder ve mühendise "Bunu gerçekleştirmek için X arayüzünü kullanacağım, Y donanım timer'ını kuracağım" şeklinde bir Markdown mimari plan sunar³⁰.
2. **Mühendis Onayı:** Siz, başmimar olarak bu planı gözden geçirir, gerekirse yönlendirmeler eklersiniz (Context injection)³⁰.
3. **TDD Tabanlı Uygulama:** Plan onaylandıktan sonra, Claude önce bu planın başarısını ölçecek Unit Testleri kodlar. O testler beklenen şekilde hata verdiğinde (red state), asıl motor veya batarya kodlarını yazarak testin geçmesini (green state) sağlar⁴⁰. Bu yöntem halüsinasyonu ve kod bozulmalarını sıfıra indirger.

Başmimar Seviyesinde Yapay Zeka Yönergeleri (Master Prompts)

Claude ile bu çapta bir projeyi yürütmek için, modelin basit bir asistan rolünden çıkarılıp "Sistem Mimarı" rolüne yerleştirilmesi gerekmektedir. Aşağıdaki promptlar, yazılım geliştirme V-Modelini baz alan ardışık iterasyonlar için tasarlanmış profesyonel araçlardır. Proje başlangıcında terminal üzerinde çalıştırılarak kullanılmalıdır.

Aşama 1: Proje Temellendirme ve Ortam Ayarları

Bu prompt, sistemin otomotiv kültürünü anlaması ve gerekli dosya ağacını inşa etmesi içindir:

Prompt 1 (Initialization):

"Şu andan itibaren 'Kıdemli Otomotiv Yazılım Baş Mimarı ve ISO 26262 Fonksiyonel Güvenlik Denetmeni' rolünü üstleniyorsun. Elektrikli (BMS odaklı), Dizel (EDC15C7, DPF, SCR kontrolü) ve Benzinli (Lambda kapalı döngü kontrolü) motorlar için AUTOSAR Classic ve Adaptive standartlarında entegre bir yazılım mimarisi geliştireceğiz.

İlk iterasyon görevimiz:

1. Bulunduğumuz dizinde, projeye uygun klasör yapısını (bsw, rte, swc, tests, arxml_definitions) bash komutlarıyla oluştur.
2. ISO 26262 V-Modeli ve UNECE R156 Siber Güvenlik standartlarını merkeze alan kapsamlı bir CLAUDE.md dosyası oluştur. Bu dosya MISRA C kısıtlamalarını, ASIL D kod standartlarını (dinamik bellek yasağı vb.), AUTOSAR iletişim prensiplerini (Runnable, Port, Interface) ve projenin TDD ile geliştirilme zorunluluğunu içermelidir.

3. Mimariyi özetleyen detaylı bir README.md hazırla ve benden onay bekle."

Aşama 2: Batarya Yönetim Sistemi (BMS) AUTOSAR Modellemesi

Sistem düzeyinden yazılım bileşeni tasarım düzeyine (Top-Down) geçiş promptudur:

Prompt 2 (Architecture Design):

"Elektrikli Araçlar için Batarya Yönetim Sistemi (BMS) Hücre Dengeleme (Cell Balancing) bileşeninin (SWC) yazılım mimarisini tasarlayacağız. Bu modül termal kaçak (thermal runaway) riskine karşı ISO 26262 ASIL D sertifikasyonuna tabidir. Lütfen kodlama yapmadan önce sadece mimari tasarım raporu sun (Plan Modunu kullan).

Rapor şunları içermelidir:

- İlgili SWC'nin AUTOSAR Application Layer içindeki konumu ve diğer modüllerle haberleşme arayüzleri (Client-Server ve Sender-Receiver Interface'ler).
- MCAL katmanındaki donanım bağımlılıkları (sıcaklık okuması için ADC konfigürasyon ihtiyacı).
- HARA analizine uygun biçimde, bir sensör arızası (Fault) durumunda devreyi kontaktörlerle kesecek olan 'Fail-Safe' durum makinesi (State Machine) tasarımı. Raporu tamamladığında onayımı iste."

Aşama 3: İçten Yanmalı Motor Uygulama Algoritması (Implementation)

Bu prompt, tasarlanmış bir mimari üzerinden C kodu üretimi ve TDD döngüsünü zorunlu kılar:

Prompt 3 (TDD Implementation):

"Benzinli motorlar için 'Lambda Kapalı Döngü Kontrolü ve Enjeksiyon Avansı' modülünü geliştireceğiz. Görevin:

- Öncelikle fonksiyonun doğru çalışıp çalışmadığını (MC/DC kapsamını %100 test edecek) doğrulayan bir Unit Test dosyası (örneğin GoogleTest veya CMock ile) yaz.
- Testleri çalıştırarak başarısız olduklarını bana terminal çıktısıyla göster.
- Ardından, MISRA C:2023 standartlarına ve AUTOSAR RTE API kullanım yönergelerine (Rte_Read(), Rte_Write()) eksiksiz uyan C kodunu (Runnable Entity) oluştur. Kodun içinde WOT (Wide Open Throttle) sensörü algılandığında açık döngü enjeksiyon haritasına geçişi sağlayan mantığı uygula.
- Son olarak kodları derle ve testlerin yeşil (pass) duruma geçtiğini kanıtla."

Aşama 4: Siber Güvenlik Analizi ve Sürekli Entegrasyon (CI)

UNECE regülasyonlarını güvence altına alan siber güvenlik denetim döngüsüdür:

Prompt 4 (Security & Validation):

"Yazdığımız modülleri ISO/SAE 21434 siber güvenlik ve ISO 26262 doğrulama (Verification) kurallarına göre analiz etmeliyiz. Şu ana kadar ürettiğin C kod dosyalarını baştan sona tara:

- Olası tampon bellek aşımı (buffer overflow), sınır dışı dizi erişimi (out-of-bounds array access) veya pointer aritmetiği hataları var mı?
- Kodun UNECE R156 SUMS (Software Update Management System) kuralları gereği, OTA güncellemesi sırasında araç hafızasında (EEPROM/Flash) kalıcı bir zafiyet bırakma ihtimalini

değerlendirir.

3. Tespit ettiğin her bir zafiyet için detaylı bir hata enjeksiyonu (fault injection) raporu sun ve koddaki güvenlik iyileştirmelerini (patch) uygula."

Sonuç ve Vizyon

Otomotiv yazılımı endüstrisinde rekabetçi bir oyuncu olmak, ticari bir mobil uygulama geliştirmekten öte, insan güvenliğini, elektromekanik donanım limitlerini ve uluslararası siber/fonksiyonel güvenlik hukuku normlarını kusursuz bir mimaride sentezlemeyi zorunlu kılar. AUTOSAR altyapısı bu sistemlerin evrensel standardını sağlarken, elektrikli, benzinli ve dizel motor kontrol stratejilerinin her biri ayrı birer mühendislik disiplindir.

Bu devasa ve karmaşık sürecin yapay zeka entegrasyonu ile yönetilmesi, geliştirme saatlerinde (man-hours) muazzam bir maliyet düşüşü yaratma potansiyeli taşır. Ancak, yapay zeka ajanlarının otonom doğasının kontrol edilemez bir kara kutu (black box) üretmemesi için; kodlama öncesi sağlam bir Model Tabanlı Tasarım (MBD) yapılması, projenin hiyerarşik bağlamının CLAUDE.md gibi mekanizmalarla AI sınırlarına kodlanması ve kesintisiz Test-Driven Development (TDD) doğrulamalarının işletilmesi elzemdir. Süreç bu raporda belirlenen profesyonel adımlar ve yönetici promptlarla disiplin altında yürütüldüğünde, sistemin TSE tip onayları ve Avrupa WP.29/UNECE sertifikasyonlarından geçebilecek kalibrede üretim yapması teknik olarak temin edilmiş olacaktır.

Alıntılanan çalışmalar

1. AUTOSAR Classic vs Adaptive: Architecture, Use Cases and Selection Guide, <https://www.embien.com/automotive-insights/autosar-classic-vs-adaptive>
2. AUTOSAR Adaptive vs Classic 2023: Standards change - Mobility Rockstars, <https://mobilityrockstars.com/en/embedded-systems-en/autosar-adaptive-vs-classic-differences-advantages-and-outlook-2023/>
3. AUTOSAR Classic vs AUTOSAR Adaptive - LDRA, <https://ldra.com/autosar/>
4. armfield - Al Farez Trading Co., http://www.alfarez.com/armfield_data/cm/CM11%20Gasoline%20Engine%20-%20Issue%202.pdf
5. Gasoline Engine Management: Systems and Components, https://www.cargeek.live/docs/Gasoline_Engine_Management_Systems_and_Components.pdf
6. Catalogo Lambda Sensors | PDF | Ignition System | Combustion - Scribd, <https://www.scribd.com/document/406715467/Catalogo-Lambda-Sensors>
7. Case Study: Engine Control Application - Uni Ulm, https://www.uni-ulm.de/fileadmin/website_uni_ulm/iui/Ulmer_Informatik_Berichte/2010/UIB-2010-03.pdf
8. CN101363382A - ECU for electric-controlled petrol engine work system - Google Patents, <https://patents.google.com/patent/CN101363382A/en>
9. Lambda load control in spark ignition engines, a new application of prechamber ignition systems | Request PDF - ResearchGate,

- https://www.researchgate.net/publication/350846127_Lambda_load_control_in_spark_ignition_engines_a_new_application_of_prechamber_ignition_systems
10. 2019-32-0535: Development of an Injection System for Small Engines with Adaptive Control for Sensorless Lambda Operation - Technical Paper - SAE Mobilus,
<https://saemobilus.sae.org/papers/development-injection-system-small-engines-adaptive-control-sensorless-lambda-operation-2019-32-0535>
 11. Nitro Data. EDC15C7 Chip Tuning, DPF-EGR-AdBlue Solution Files,
<https://nitrodata.com.tr/tr/edc15c7-chip-tuning-dpf-egr-adblue-solution-files-nitro-data>
 12. Apply ISO 26262 and ASIL levels - Arm Learning Paths,
https://learn.arm.com/learning-paths/automotive/openadkit2_safetyisolation/1c_is_o26262/
 13. BMS Algorithms Explained: SOH Estimation, SoP, SoE, Cell,
<https://sunlithenergy.com/bms-algorithms-explained/>
 14. The AUTOSAR Classic vs Adaptive – What a Model-Based Developer should know about it, <https://www.btc-embedded.com/autosar-classic-vs-adaptive>
 15. Differences Between Autosar Classic vs. Autosar Adaptive - New Digital Street,
<https://newdigitalstreet.com/autosar-classic-vs-autosar-adaptive/>
 16. What Is ASIL? A Guide to Automotive Safety Integrity Levels in ISO 26262 - Jama Software,
<https://www.jamasoftware.com/requirements-management-guide/automotive-engineering/asil/>
 17. Eindhoven University of Technology MASTER Software emulation of STM32 controller for virtual embedded design/test environment Jo,
https://pure.tue.nl/ws/portalfiles/portal/138967031/1377566_MScThesisChandrika_Joshi_submission.pdf
 18. ISO 26262 Part 6 – Software Development Complete Guide - PiEmbSysTech,
<https://piembstech.com/iso-26262-part-6-software-development/>
 19. Comparison of the Deviations between MiL, SiL and HiL Testing - OPUS,
<https://opus4.kobv.de/opus4-haw/files/870/I000784520Thesis.pdf>
 20. An Analysis of ISO 26262: Using Machine Learning Safely in Automotive Software,
https://esnl.hnu.edu.cn/ecps2018/Using_Machine_Learning_Safely_in_Automotive_Software.pdf
 21. What is ISO 26262? | A Guide to Functional Safety Standard - Embitel,
<https://www.embitel.com/automotive-insights/what-is-iso-26262>
 22. ISO 26262 Software Compliance in the Automotive Industry - Parasoft,
<https://www.parasoft.com/learning-center/iso-26262/what-is/>
 23. Otonom Kamyon Nasıl Çalışıyor? Sensörlerden Yapay Zekya, Seviye 4 Teknolojisinin Anatomisi,
<https://www.kamyonum.com.tr/haber/Otonom-Kamyon-Nasil-Calisiyor-Sensorlerden-Yapay-Zekya-Seviye-4-Teknolojisinin-Anatomisi/46787>
 24. ISO 21434 services - Bureau Veritas Cybersecurity,
<https://cybersecurity.bureauveritas.com/services/iot/automotive/iso-21434-services>

25. Otonom Araçlar Mevcut Durum Analizi Raporu - Haberleşme Genel Müdürlüğü,
<https://hgm.uab.gov.tr/uploads/pages/akilli-ulasim-sistemleri-aus/otonom-araclar-mevcut-durum-analizi-raporu.pdf>
26. ISO/SAE 21434 ve UNECE R155: Bağlı Filolarda Araç Siber,
<https://aractakipjet.com/blog/iso-sae-21434-unece-r155-arac-siber-guvenlik-filo-2026/>
27. TREDER'den ECE R156 ve SUMS Eğitimi,
<https://treder.org.tr/trederden-ece-r156-ve-sums-egitimi>
28. ISO/SAE 21434 vs UNECE R155: What's the Difference? | Agnile,
<https://agnile.com/blog/iso-21434-vs-unece-r155>
29. A Comparative Analysis of UNECE WP.29 R155 and ISO/SAE 21434 - CNR-IRIS,
https://iris.cnr.it/retrieve/8ce973fd-a139-4021-b060-37c2124f8567/prod_474034-doc_193290.pdf
30. Context Engineering: Make AI Coding Tools Actually Work - Collin Wilkins,
<https://collinwilkins.com/articles/context-engineering-ai-coding-tools>
31. Claude Code vs Roo Code in 2026 | Build This Now,
<https://www.buildthisnow.com/blog/tools/extensions/claude-code-vs-roo-code>
32. Aider vs Cline 2026: Open-Source AI Coding Tools Compared - Morph,
<https://www.morphllm.com/comparisons/aider-vs-cline>
33. eltociear/awesome-AI-driven-development - GitHub,
<https://github.com/eltociear/awesome-AI-driven-development>
34. Best practices for Claude Code, <https://code.claude.com/docs/en/best-practices>
35. ai-toolkit - AI Agents on GitHub | SkillsLLM, <https://skillsllm.com/skill/ai-toolkit>
36. Roo Code: What Practitioners Actually Found — Enterprise DNA,
<https://enterprisedna.co/resources/blog/practitioner-roo-code-review>
37. 10 Claude Code Best Practices May 2026 - Willow Voice,
<https://willowvoice.com/blog/10-essential-claude-code-best-practices>
38. CLAUDE.md Best Practices - Medium,
https://medium.com/@onix_react/claude-md-best-practices-a60d726dc2a5
39. CLAUDE.md Best Practices - YouTube,
<https://www.youtube.com/watch?v=x5WWFL0nlqk>
40. Claude Code Best Practices: Planning, Context Transfer, TDD - DataCamp,
<https://www.datacamp.com/tutorial/claude-code-best-practices>
41. Claude Code: Best Practices for Developers - SAP Community,
<https://community.sap.com/t5/artificial-intelligence-blogs-posts/claude-code-best-practices-for-developers/ba-p/14394164>